

Technical Report: Garbage Collected Virtual Machine

Course Code: COD7001

Mayur Kamble
2025MCS2116

Nizaul Rahman
2025MCS2099

1 Introduction

The objective of this project was to develop a Virtual Machine (VM) capable of autonomous memory management. The primary challenge involved implementing a Mark-and-Sweep Garbage Collector that remains stable under high allocation pressure, handles circular references, and avoids the common pitfalls of C stack limitations.

2 Architectural Design

To overcome segmentation fault issues associated with large-scale data processing in C, the following architectural decisions were made:

2.1 Heap-Based VM Runtime

The entire VM structure, including its 1,048,576-slot operand stack, is allocated on the heap using `malloc`. This bypasses the default 8MB Linux system stack limit, enabling industrial-scale object management.

2.2 Iterative Worklist Marking

Instead of recursive traversal (which risks stack overflow on deep object graphs), the marking phase uses a manual, heap-allocated worklist to perform graph traversal safely.

2.3 Linked-List Object Tracking

All allocated objects are tracked via a global linked list, enabling a linear-time sweep phase independent of object complexity.

3 Garbage Collection Logic

The VM implements a robust Mark-and-Sweep garbage collection algorithm with adaptive thresholding:

3.1 Mark Phase

The collector scans the root set, which includes:

- All objects on the operand stack
- Objects stored in the global memory array

A depth-first traversal is performed using an explicit worklist.

3.2 Cycle Safety

By checking the `marked` flag before traversing references, the algorithm is immune to infinite loops caused by circular references (e.g., mutually referencing objects).

3.3 Sweep Phase

The VM traverses the global linked list and frees memory for every object where:

`marked = false`

3.4 Self-Scaling Threshold

The garbage collection trigger dynamically adjusts according to:

`maxObjects = currentObjects × 2`

This approach balances performance with memory usage under varying workloads.

4 Test Results & Analysis

The implementation was subjected to a rigorous test suite, validating correctness and stability.

4.1 Functional Verification (`gc_test.bin`)

Objective: Confirm collection of unreachable objects.

Log Output: GC: Collected 3 objects. 1 remaining.

Analysis: The VM correctly identified and reclaimed temporary objects while preserving the final live value on the stack.

4.2 Circular Reference Test (`cycle_test.asm`)

Objective: Validate handling of self-referential structures.

Log Output: GC: Collected 0 objects. 4 remaining.

Analysis: The VM safely traversed circular references without crashing or hanging, correctly identifying all objects as reachable.

4.3 Stress Test & Adaptive Scaling (stress_test.bin)

Objective: Evaluate performance under heavy allocation pressure (100k+ objects).

Log Output:

GC: Collected 1024 objects. 0 remaining. (Multiple cycles)

GC: Collected 0 objects. 673 remaining.

Analysis: The adaptive trigger logic functioned as expected, reclaiming thousands of temporary objects while scaling thresholds dynamically as live object counts changed.

5 Memory Safety Verification (Valgrind)

To ensure professional-grade memory correctness, the VM was analyzed using Valgrind Memcheck.

- **Result:** All heap blocks were freed — no leaks detected
- **Error Summary:** 0 errors from 0 contexts

6 Instruction Set Architecture (ISA)

The following opcodes are synchronized between the Python assembler and the C-based VM:

Opcode	Mnemonic	Function
0x01	PUSH	Allocate OBJ_INT and push to stack
0x03	DUP	Duplicate the top stack element
0x30	PAIR	Create a nested OBJ_PAIR structure
0x32	STORE	Move a stack object to global storage
0x23	GC	Manually trigger garbage collection
0xFF	HALT	Terminate execution and free all memory

7 Conclusion

The Virtual Machine successfully fulfills the objectives of Lab 5. By leveraging heap-based runtime allocation and an iterative Mark-and-Sweep garbage collector, the system provides a stable environment for memory-intensive workloads. The zero-leak Valgrind report confirms that the garbage collector is both accurate and exhaustive, demonstrating correctness, robustness, and production-level memory safety.